

# Requirements Engineering for the AI Age

---

A step-in-ready framework for governing requirements as living, machine-readable, enforceable engineering artifacts for real-world application.

Jeffrey Wallk



STAMP  
SHACL  
SWRL  
RDSG

**v1.0**

# Challenges with Requirements Management

Here are a few key takeaways — and what this framework answers.

**"Requirements disappear  
after the system is built"**

Every req lives as a typed ontology individual — versioned, traceable, machine-readable forever

**"Terminology conflict  
kills interoperability"**

Core canonical vocabulary + domain extensions, with automated disambiguation at every gateway interaction

**"AI autonomy worries  
me in this discipline"**

Human authority is enforced by architecture — no irreversible action without explicit, verified human approval

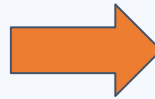
**"We don't want to be  
a talking shop"**

One concrete deliverable per 90-day cycle. Each of you has a named contribution. Work starts today.

# The shift we are trying to achieve.

## Requirements Management

- Documents stored in separate tools
- Forgotten after system is built
- No machine-readable structure
- Traceability is manual and brittle
- Ambiguity managed by convention
- AI has no awareness of requirements



## Requirements Engineering

- Requirements are ontology individuals in a triplestore
- Active throughout the system lifecycle
- Machine-readable, formally typed, versioned
- Full traceability — automated and queryable
- Ambiguity is detected and resolved by the framework
- AI operates inside a requirement-defined control envelope

# Three premises. Non-negotiable.

These are the principles the framework is built on — and what our working group is committed to.

01

## Requirements are data, not documents.

A requirement is an ontology individual with typed properties — persona, job, outcome, constraint, acceptance criterion. It lives in a triplestore, is enforced by shapes, activates rules, and is traceable to every AI action it governs. If it only exists in a PDF, it does not exist.

02

## Ambiguity is a system hazard.

Terminology inconsistency across domains does not just slow things down — it produces unsafe control actions. STPA (Leveson) classifies it as a formal hazard. Our framework detects it at ingestion, flags it for resolution, and blocks any AI output that contains unresolved terminology before it reaches a user.

03

## Governance without enforcement is decoration.

A requirements standard that cannot be automatically evaluated is advice, not governance. Every requirement in this framework has machine-executable acceptance criteria. The AI cannot produce an output that violates them. The framework either enforces the constraint or surfaces the gap — there is no silent non-compliance.

# From concern to contribution — the direct map.

| Your concern from Meeting 1                              | Framework mechanism                                                                                     | How you contribute                                                        |
|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| "Terminology tribalism across our domains"               | Canonical ontology + domain extensions + disambiguation barrier detection                               | Bring 3 contested terms from your domain. We model them together.         |
| "AI changes are hard to trace"                           | Requirements Findings log + RequirementsReview trigger after 3 failures                                 | Define what a traceability failure looks like in your domain's context    |
| "We need interoperability, not just standards"           | SHACL shapes enforce cross-domain constraints at the gateway boundary                                   | Identify one interoperability constraint from your domain to encode       |
| "Requirements need to be engineerable, not just managed" | Stage 1 elicitation pipeline: 4 questions convert any requirement to a typed ontology individual        | Answer the 4 questions for one real requirement from your current project |
| "AI shouldn't have autonomous authority here"            | Simplex Action Gateway: irreversible actions require verified human approval — architecturally enforced | Identify one irreversible action in your domain that needs this gate      |

# The gateway in plain language — no ontology background required.

*Think of the gateway as an engineering discipline applied to every AI interaction.*

1

**Check who is asking  
and what job they  
need done**

Match the request to  
known requirements  
for this person's role  
and context

2

**Resolve every entity  
to its canonical form**

Eliminate terminology  
ambiguity before it  
enters the system

3

**Evaluate all  
applicable  
constraints**

SHACL shapes test  
structural  
completeness,  
compliance citations,  
and safety rules

4

**Apply behavioral  
governance rules**

SWRL rules inject  
outcome metrics,  
detect conflicts,  
enforce timing and  
sequencing

5

**Authorize and  
compile  
the directive**

Human approval  
required for  
irreversible actions  
before any execution

The AI can only respond within the control envelope these five steps define. Every output is validated before delivery. Every irreversible action requires a verified human decision.

# Your entry point: four questions that do the engineering for you.

No ontology expertise required. These questions translate any practitioner's knowledge directly into a governed, machine-readable requirement.

## "Who are you?" → Persona

Your role, domain, and responsibility context. This scopes which requirements apply, which compliance obligations are active, and what data you're authorized to see.

*"I am a systems safety engineer responsible for brake certification on regional rail platforms."*

## "What are you trying to accomplish?" → Functional Job

The job-to-be-done — the progress you are trying to make. Not a task, but the outcome. This anchors every downstream constraint and success criterion.

*"I need to verify that all brake system requirements satisfy EN 50128 SIL 3 criteria before the design review."*

## "What does success look like?" → Desired Outcome

A measurable metric for completion. This becomes the AI's mandatory success criterion — it cannot deliver a response that does not address your outcome measure.

*"Complete coverage of all 47 safety-critical requirements against SIL 3 acceptance criteria within 4 hours."*

## "What gets in your way?" → Job Barrier

The friction point that prevents you from getting this job done. This triggers the framework's disambiguation and hazard detection for that specific barrier type.

*"Requirements are written in three different terminologies — project, supplier, and certification body — with no canonical mapping."*

# Worked example: one requirement, traced end-to-end.

Domain: Rail Safety

*"The braking system shall achieve a minimum deceleration of 1.1 m/s<sup>2</sup> within 3 seconds of activation under full load conditions in accordance with EN 13452-1."*

## Stage 0 Resolution

Persona:  
SafetyEngineer\_Rail  
Job:  
VerifyBrakingRequirements  
Compliance: EN  
13452-1, SIL 3

## Step 1 Ontology

"Deceleration" →  
bk:DecelerationRate  
"Full load" →  
bk:MaximumLoadCondition  
"EN 13452-1" →  
reg:EN13452\_1\_Cert

## Step 2 SHACL

✓ bk:DecelerationRate  
has numeric threshold  
✓ EN 13452-1 citation  
resolvable  
⚠ "full load" —  
terminology barrier  
flagged

## Step 3–4 Rules + PEP

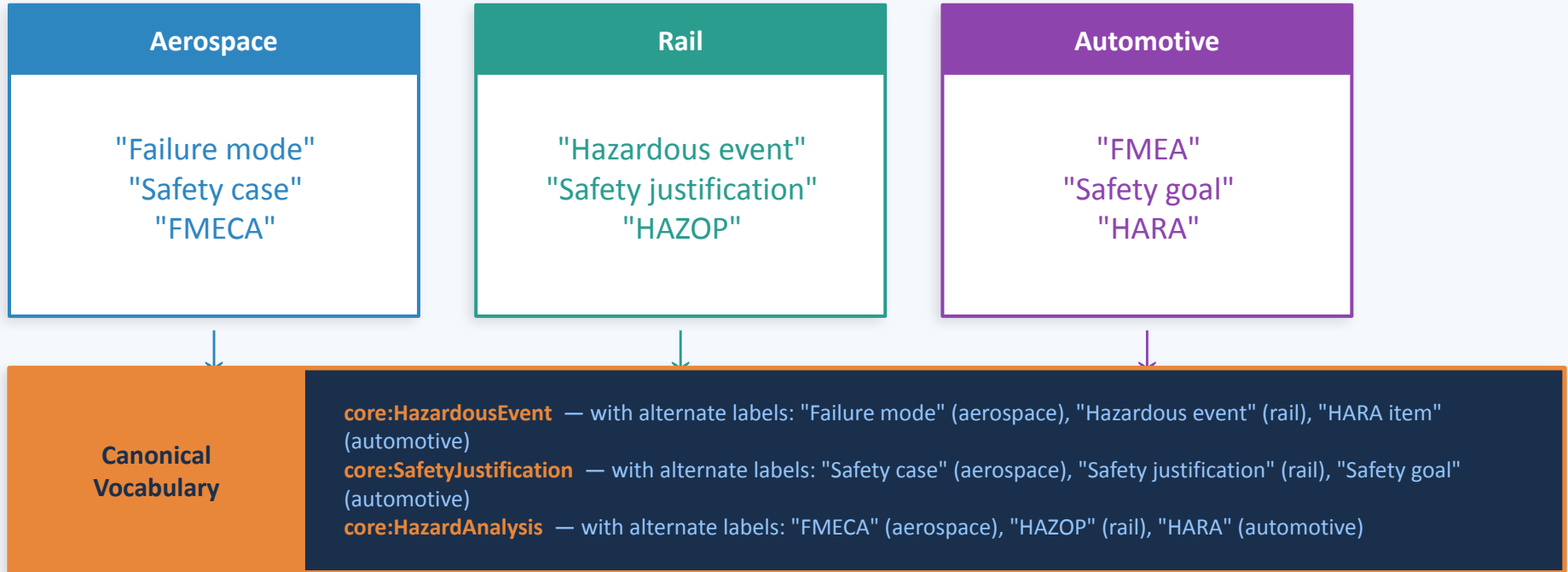
REQ-001: Inject 1.1  
m/s<sup>2</sup> as success metric  
REQ-007: 3-second SLA  
becomes NFR threshold  
TMP-001: Timeout  
armed at 3s

## Service Facade Validation

✓ 1.1 m/s<sup>2</sup> addressed  
in response  
✓ EN 13452-1 cited  
✗ "full load"  
unresolved → UCA-002  
recycle

UCA-002 fired on "full load" terminology inconsistency — agent recycled with disambiguation instruction. EN 13452-1 mandatory compliance chunk pre-fetched and prepended to agent context. Final response delivered only after all AcceptanceCriteria pass.

# Terminology tribalism — solved architecturally.



The framework preserves your domain's vocabulary as alternate labels. You never lose your terminology — you gain shared resolution. Every AI output uses the canonical form; every human sees their familiar term.

# AI as a governed tool — not an autonomous agent.

Every concern about AI autonomy in this discipline is addressed by architecture, not by trust.

## Human governance layer

Humans define all rules

Domain stewards activate requirements. STPA analysts define unsafe control actions. Compliance officers validate constraints. Human authority is the source of all policy.

## Semantic gateway

Requirements as the lens

Every AI interaction governed by machine-executable constraints derived from human-defined requirements. AI never sees a raw prompt — it sees a governed directive.

## Service facade (Zone 4)

10-criterion validation gate

Every AI response evaluated against structural completeness, ontology consistency, compliance citations, scope boundary, and UCA safety checks before reaching any human.

## AI agents + vector DBs (Zone 5)

The controlled process

AI operates inside the directive — cannot exceed scope, violate authorization, or produce unresolved terminology. Monitored for timeout and scope overflow in real time.

## Simplex action gateway (Zone 6)

Irreversibility enforcement

Regulatory filings, financial transactions, data deletions require a verified human approval token. No emergency bypass exists. Every action logged immutably.

**The AI is the controlled process. The requirement-defined control structure governs its behavior. Human authority is enforced at every layer — not assumed.**

# Where you plug in — every expertise has a home.

## Domain practitioner (Aerospace · Rail · Automotive)

Stage 1 Elicitation  
Stage 0 Persona  
Registry

You supply the requirements, the domain constraints, and the real-world failure modes that feed Stage 1 elicitation. You are the requirements source — the framework formalizes what you already know.

## Systems engineer (MBSE · Architecture)

Step 2 SHACL  
Step 3 SWRL

You define the control structure relationships — which requirements govern which system functions, how JTBD hierarchies map to system decomposition, and how STPA UCAs connect to design constraints.

## Ontologist / knowledge engineer

Zone 3 Ontology  
+ Rules Store

You author the canonical vocabulary, design the domain extensions, and build the SHACL shapes and SWRL rules that enforce the framework's behavioral governance.

## Safety analyst (STPA · HAZOP · FMEA)

UCA Profile  
Simplex Gateway

You define Unsafe Control Actions for each job, set timeout thresholds, specify safe-state actions, and classify Zone 6 actions by reversibility. Your analysis is the framework's hazard model.

## Compliance / domain steward

You activate and deprecate requirements, review governance gaps, validate ComplianceRequirement linkages, and authorize the RequirementsReview process when acceptance criteria fail repeatedly.

# Initial Set of Concrete Deliverables

D1

## Requirements Vocabulary Registry v1.0

Five contested terms from each participant's domain, mapped to canonical core vocabulary with domain-specific alternate labels. Hosted in shared repository. Machine-readable OWL/SKOS format.

 All members contribute 5 terms each ·  Meeting 2

D2

## Worked Example Library

Three end-to-end requirement traces — one from aerospace, one from rail, one from automotive — through the full gateway pipeline. Demonstrates the framework using real domain requirements.

 Domain practitioner volunteers ·  Meeting 3

D3

## Stage 1 Elicitation Template

A practitioner-facing form implementing the four questions plus five STPA UCA elicitation questions. Usable without ontology expertise. Available as a Word template and a structured JSON schema.

 Steward + STPA analyst ·  Meeting 3

D4

## INCOSE 2026 Interoperability Position Paper

A 4-page position paper on Requirements Engineering as the semantic foundation for system-of-systems interoperability, submitted to the 2026 INCOSE initiative. Based on framework work.

 Jeffrey leads, all contribute ·  Meeting 4

*These are small, completable, and directly useful. None requires the full framework to be implemented first. Each one builds the foundation for the next.*

# Before next meeting — some homework

These are not homework. They are contributions that make this real. Bring your answers to the next session.

## Domain practitioners (aerospace · rail · automotive)

Name five terms from your domain that mean different things to different stakeholders. No formal preparation needed — just bring the list.

*These seed D1: the Vocabulary Registry and demonstrate the tribalism problem concretely.*

## Systems / safety engineers

Identify one interoperability failure from your experience — a case where requirements from two different systems, suppliers, or standards collided. One paragraph.

*These become the failure-mode library that justifies the STPA UCA analysis in the framework.*

## Ontologists / knowledge engineers

Propose one SHACL shape — even informally, in plain language — that would have prevented a real requirements failure you have seen.

*These become the first ControlRequirements in the shared SHACL store: D1 enforcement rules.*

## Stewards / compliance leads

Answer: what would a practical framework deliverable need to look like for your organization to adopt it? Format, hosting, governance, access model — be specific.

*This defines our D1–D4 delivery constraints before we build instead of after.*

## All participants

Answer the four elicitation questions for ONE real requirement from your current project. Use the template on the previous slide. Bring it to the next session.

*Fastest path from conversation to working framework. One requirement from each of you = our first corpus.*

# One commitment.

We leave this room having agreed to one concrete, completable, useful output — with a date attached.

---



Magdalena: Repository live before next invitation goes out



Jeffrey: Elicitation template in shared repository by end of week



All members: Five terms + one real requirement for next session



Group: Name the one 90-day deliverable we commit to building together

**The problem is real. The urgency is real. The expertise is in this room. Let's build.**